PROJECT REPORT ON

HOUSE PRICE PREDICTION USING MACHINE LEARNING

Submitted By:

Uplakshya Tiwari

Ujjval Dubey

Tejas Pandey

Under the Guidance of:
Mr. Arpit Dubey



Acknowledgement

We would like to express our profound gratitude to everyone who supported us in completing this project.

We are deeply thankful to our guide, Mr. Arpit Dubey, for their continuous encouragement, invaluable insights, and technical guidance throughout the development of this project. Their mentorship was instrumental in shaping my understanding of Machine Learning concepts and ensuring the successful execution of this work.

Lastly, we extend our heartfelt thanks to our friend, and classmates for their unwavering support and motivation during the course of this project.

Uplakshya Tiwari

Ujjval Dubey

Tejas Pandey

Abstract

The real estate market is highly dynamic and volatile, making the accurate prediction of house prices a challenging yet essential task for buyers, sellers, and investors. This project aims to build a robust Machine Learning system to predict the median house values in various districts using demographic and geographical data. The problem statement focuses on determining how different features such as location (latitude and longitude), median income, total rooms, and proximity to the ocean influence the pricing of a house.

The study utilizes a housing dataset comprising 20,640 records and 10 attributes. The methodology involves an extensive Exploratory Data Analysis (EDA) to understand feature distributions, identify outliers, and detect missing values. A critical preprocessing pipeline was engineered using ColumnTransformer to perform median imputation and standard scaling for numerical features, and frequent value imputation alongside one-hot encoding for categorical variables. Data leakage was strictly prevented by fitting the imputers and scalers only on the training data after an 80-20 train-test split.

To achieve the objective, three Machine Learning algorithms were implemented and compared: Linear Regression (as a baseline model), Random Forest Regressor, and eXtreme Gradient Boosting (XGBoost) Regressor. The models were evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 Score. The XGBoost model outperformed the others, achieving the highest R^2 score of 0.821 and the lowest RMSE, proving its high reliability in capturing the complex, non-linear relationships in the dataset. The final outcome is a scalable and highly accurate predictive pipeline that can be utilized in real-world real estate applications to estimate property values efficiently.

Table of Contents

1. Chapter 1: Introduction

1.1 Background

1.2 Problem Statement

1.3 Need for House Price Prediction

1.4 Objectives

1.5 Scope

1.6 Real-world Applications

2. Chapter 2: System Overview

2.1 Complete Workflow of the Project

2.2 Dataset Description

2.3 Features and Target Variable

2.4 Input and Output

2.5 System Architecture

3. Chapter 3: Methodology

3.1 Importing Libraries

3.2 Loading the Dataset

3.3 Dataset Exploration and EDA

3.4 Handling Missing Values

3.5 Removing Duplicates and Statistical Summary

3.6 Train-Test Split

3.7 Data Preprocessing Pipeline

4. Chapter 4: Machine Learning Models

4.1 Linear Regression (Baseline Model)

4.2 Random Forest Regressor

4.3 XGBoost Regressor

5. Chapter 5: Results and Discussion

5.1 Model Evaluation Metrics

5.2 Comparison of Models

5.3 Best-Performing Model and Interpretation

6. Chapter 6: Conclusion and Future Scope

6.1 Summary of Work

6.2 Achievements

6.3 Limitations

6.4 Future Improvements

7. References

8. Appendix

Chapter 1: Introduction

1.1 Background

The real estate industry plays a pivotal role in the global economy. For decades, the valuation of properties was primarily done through manual appraisals, which are time-consuming, expensive, and subject to human bias. With the advent of data science and Machine Learning, predictive modeling has emerged as a powerful tool to automate and optimize property valuation. By analyzing historical housing data, algorithms can identify hidden patterns and trends to forecast property prices accurately.

1.2 Problem Statement

Housing prices are influenced by a multitude of intersecting factors, ranging from geographical coordinates (latitude and longitude) to socio-economic indicators (median income of the neighborhood). The core problem is to design an automated predictive model that can take these heterogeneous data points as inputs and output a continuous numerical value representing the median house price. The model must handle missing data, categorical text data, and variations in feature scales to ensure accurate predictions.

1.3 Need for House Price Prediction

- For Buyers: To ensure they are not overpaying for a property.
- For Sellers: To optimally price their property to attract buyers while maximizing profit.
- For Real Estate Agents & Investors: To identify undervalued properties and make data-driven investment decisions.
- For Financial Institutions: To assess the value of collateral before approving mortgage loans.
-

1.4 Objectives

- To perform comprehensive Exploratory Data Analysis (EDA) to understand the distribution of housing data.
- To design a robust data preprocessing pipeline that prevents data leakage.
- To train multiple Machine Learning models (Linear Regression, Random Forest, XGBoost).
- To evaluate and compare the models using standard regression metrics to determine the most effective algorithm.

1.5 Scope

The scope of this project is limited to the provided California housing dataset. The project covers data cleaning, feature engineering, pipeline creation, model training, and performance evaluation. It does not include web deployment or real-time data fetching, focusing purely on the core Machine Learning predictive analysis.

1.6 Real-world Applications

- Automated Valuation Models (AVMs): Systems like Zillow's Zestimate use similar principles.
- Urban Planning: Helping city planners understand affordability across different districts.
- Economic Forecasting: Tracking real estate trends as a metric for overall economic health.

Chapter 2: System Overview

2.1 Complete Workflow of the Project

The project follows a standard Machine Learning lifecycle:

1. Data Ingestion: Loading the raw CSV dataset into a Pandas DataFrame.
2. Data Exploration: Understanding shapes, data types, missing values, and statistical properties.
3. Data Visualization: Plotting distributions of categorical and continuous variables to identify skews and outliers.
4. Data Splitting: Separating the data into training and testing sets before preprocessing to ensure strict model validation.
5. Data Preprocessing: Building an automated pipeline using Scikit-Learn's ColumnTransformer to impute missing values, scale numbers, and encode text.
6. Model Training: Fitting Linear Regression, Random Forest, and XGBoost models to the processed training data.
7. Model Evaluation: Testing the models on unseen data and calculating error metrics.

2.2 Dataset Description

The dataset used in this project is a historical housing dataset containing information about various block groups (districts) in California.

- Total Records (Rows): 20,640
- Total Attributes (Columns): 10

2.3 Features and Target Variable

The dataset contains both numerical and categorical data.
Numerical Features:

8. longitude: A measure of how far west a house is.
9. latitude: A measure of how far north a house is.
10. housing_median_age: Median age of a house within a block.
11. total_rooms: Total number of rooms within a block.
12. total_bedrooms: Total number of bedrooms within a block.
13. population: Total number of people residing within a block.
14. households: Total number of households (groups of people residing within a home unit).
15. median_income: Median income for households within a block (measured in tens of thousands of US Dollars).

Categorical Features:

- ocean_proximity: Location of the house relative to the ocean (e.g., <1H OCEAN, INLAND, NEAR OCEAN, NEAR BAY, ISLAND).

Target Variable:

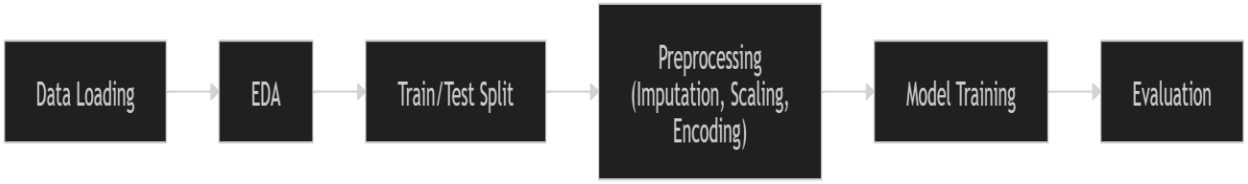
- median_house_value: The median house value for households within a block (measured in US Dollars). This is the dependent variable our models will attempt to predict.

2.4 Input and Output

- Input: A vector of 9 features (8 numerical, 1 categorical) representing a specific housing block.
- Output: A single continuous numerical value predicting the median_house_value.

2.5 System Architecture

The architecture relies heavily on Scikit-Learn's Pipeline object, ensuring that data flows sequentially from raw input to predicted output without manual intervention at each step.



Chapter 3: Methodology

This chapter details the exact step-by-step implementation as executed in the project's Jupyter Notebook.

3.1 Importing Libraries

The project begins by installing and importing essential Python libraries:

- pandas and numpy: For data manipulation, numerical computations, and DataFrame management.
- matplotlib.pyplot and seaborn: For creating visual plots and exploratory data analysis.
- sklearn.model_selection (train_test_split, etc.): For splitting the dataset.
- sklearn.pipeline and sklearn.compose (Pipeline, ColumnTransformer): For creating organized preprocessing workflows.
- sklearn.preprocessing (StandardScaler, OneHotEncoder): For feature scaling and categorical encoding.
- sklearn.impute (SimpleImputer): For handling missing data.
- xgboost and sklearn model classes: For the Machine Learning algorithms.
- sklearn.metrics: To calculate errors (MAE, RMSE, R^2).

Importance: Importing libraries sets up the development environment, granting access to optimized, pre-written mathematical functions.

3.2 Loading the Dataset

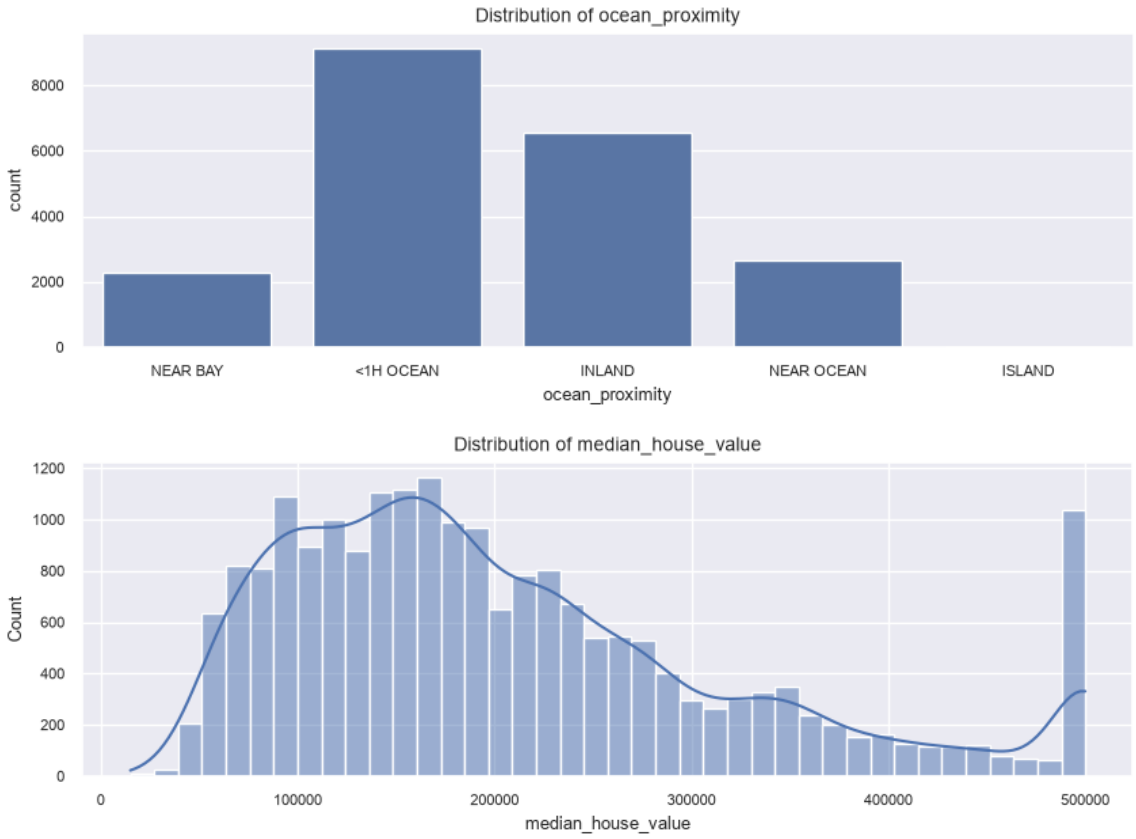
The dataset is loaded using Pandas: pd.read_csv("housing.csv"). The target column is explicitly defined as a constant TARGET_COL = "median_house_value". Configurations are applied to Pandas and Seaborn to format the display of floating-point numbers and grid styles.

3.3 Dataset Exploration and EDA

Exploratory Data Analysis was conducted to understand the dataset's characteristics.

- Shape: df.shape revealed 20,640 instances and 10 columns.
- Column Separation: The columns were programmatically split into num_cols (numerical) and cat_cols (categorical) using select_dtypes.

- Data Visualizations:
  - sns.countplot was used to plot the frequency of categories in ocean_proximity. The majority of houses were found to be in the <1H OCEAN category, followed by INLAND.
  - sns.histplot was used on the target variable (median_house_value). The histogram revealed a right-skewed distribution with a sudden, abnormal peak (higher cap) at $500,001. This indicated that values were capped, a crucial finding because it means the data is not perfectly clean and the model might struggle with houses priced above this limit.



3.4 Handling Missing Values

Using df.isna().sum(), it was discovered that the total_bedrooms column had 207 missing values.

What was done: Instead of naively replacing these null values immediately, the decision was made to handle them dynamically within the preprocessing pipeline using SimpleImputer.

Why it was done: Calculating the median on the entire dataset and replacing missing values before splitting causes Data Leakage (where information from the testing set bleeds into the training set). By deferring this to the pipeline, the median is calculated only on the training data and then applied to both sets.

3.5 Removing Duplicates and Statistical Summary

- Duplicates: df.duplicated().sum() showed 0 duplicate rows.
- Descriptive Statistics: df.describe().T was used to analyze the minimum, maximum, mean, standard deviation, and quartiles of numerical features.

Why it is important: It helps identify outliers and skewness. For instance, if a feature's mean is significantly greater than its median (50th percentile), the data is skewed, justifying the later need for feature scaling.

3.6 Train-Test Split

The data was split into independent variables (X) and the dependent variable (y). train_test_split(X, y, test_size=0.2, random_state=42)

What was done: 80% of the data (16,512 rows) was assigned to the training set, and 20% (4,128 rows) was hidden away as the testing set.

Why it is important: Splitting the data ensures that the model is evaluated on unseen data, providing an accurate measure of how the model will perform in the real world.

3.7 Data Preprocessing Pipeline

A complex preprocessing pipeline was built using ColumnTransformer to handle numerical and categorical columns differently.

Numerical Pipeline:

16. SimpleImputer(strategy='median'): Fills missing values in total_bedrooms with the median value of the training data.
17. StandardScaler(): Standardizes features by removing the mean and scaling to unit variance. This ensures that features like median_income (small numbers) and population (large numbers) are on the same scale.

Categorical Pipeline:

18. SimpleImputer(strategy='most_frequent'): Acts as a fallback for any missing categorical data by replacing it with the mode.
19. OneHotEncoder(handle_unknown='ignore'): Converts the text-based ocean_proximity into numerical format by creating binary columns for each category (e.g., 1 if INLAND, 0 otherwise).

Column Transformer:
These two pipelines were combined into a single preprocess object.

Why it is important: The pipeline ensures that raw data can be fed into the model, and all mathematical transformations are applied automatically, reliably, and without data leakage.

Chapter 4: Machine Learning Models

Once the preprocessing pipeline was established, it was integrated with various Machine Learning estimators to create the final models. Only the models explicitly implemented in the notebook are discussed below.

4.1 Linear Regression (Baseline Model)

Working Principle: Linear Regression attempts to model the relationship between two or more explanatory variables and a continuous target variable by fitting a linear equation (a straight line or hyperplane) to observed data. The goal is to minimize the residual sum of squares between the observed targets and the predictions.

Advantages: Highly interpretable, fast to train, and serves as an excellent baseline.

Disadvantages: Assumes a linear relationship; performs poorly on complex, non-linear data.

Why it was selected: It was selected as the "Baseline Model" to set a benchmark performance metric that more complex models must beat.

Prediction Method: It predicts the house price by assigning a learned weight (coefficient) to each feature (e.g., income, rooms) and summing them up along with a bias term.

4.2 Random Forest Regressor

Working Principle: Random Forest is an ensemble learning method. It constructs a multitude of Decision Trees during training. Instead of relying on a single tree, it outputs the average prediction of all individual trees. In the notebook, it was instantiated with n_estimators=100 (100 trees).

Advantages: Reduces overfitting compared to single decision trees, handles non-linear data exceptionally well, and does not require extensive feature scaling.

Disadvantages: Computationally expensive and slower to predict than linear models; acts as a "black box" with lower interpretability.

Why it was selected: To capture the complex, non-linear geographical and socio-economic relationships in the California housing data that Linear Regression missed.

Prediction Method: 100 different decision trees predict a house's price, and the Random Forest averages these 100 prices to give the final output.

4.3 XGBoost Regressor (eXtreme Gradient Boosting)

Working Principle: XGBoost is an advanced implementation of gradient boosting algorithms. Unlike Random Forest, which builds trees independently, XGBoost builds trees sequentially. Each new tree corrects the residual errors made by the previous trees. The model was configured with hyperparameters: learning_rate=0.1, max_depth=6, and n_estimators=100.

Advantages: Extremely high performance, built-in regularization to prevent overfitting, and highly optimized for speed.

Disadvantages: Highly sensitive to hyperparameters and prone to overfitting if not tuned correctly.

Why it was selected: It is widely considered the state-of-the-art model for structured tabular data.

Prediction Method: It combines weak learners (shallow trees) into a strong learner by continuously optimizing a loss function using gradient descent, yielding a highly accurate house price.

Chapter 5: Results and Discussion

After training the models via the defined pipelines, their performance was evaluated on both the training and testing datasets.

5.1 Model Evaluation Metrics

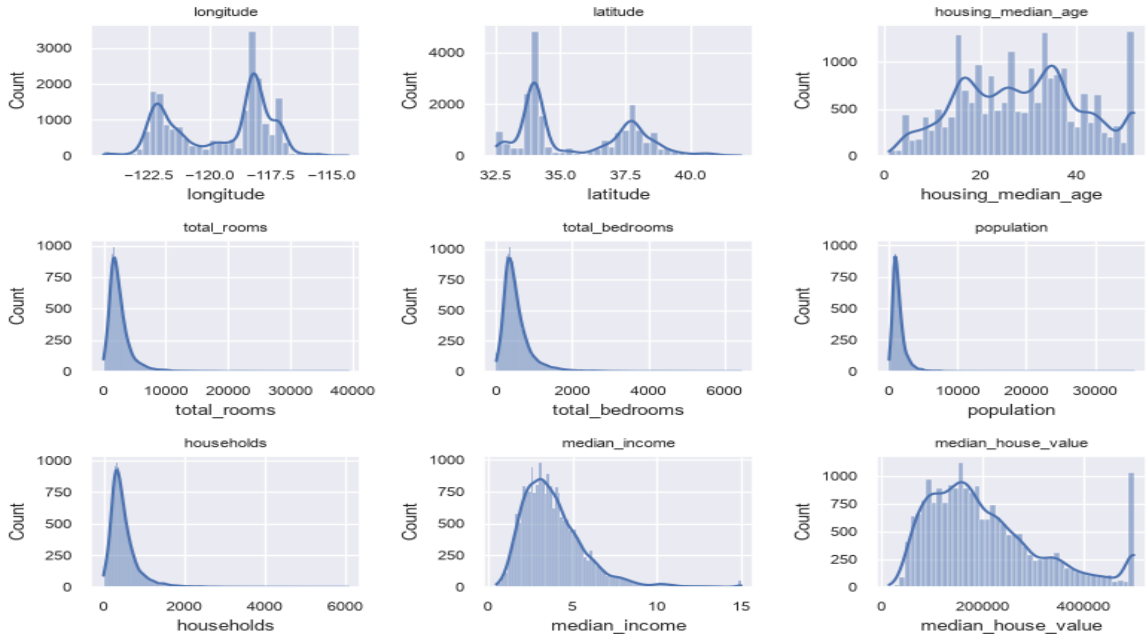
To quantify the performance, three statistical metrics were used:

- Root Mean Squared Error (RMSE): Represents the square root of the variance of the residuals. It heavily penalizes large errors.
- Mean Absolute Error (MAE): The average of the absolute differences between predictions and actual values. It gives a linear idea of the error in dollars.
- R^2 Score (Coefficient of Determination): Represents the proportion of the variance in the dependent variable that is predictable from the independent variables. A score closer to 1.0 indicates a perfect fit.

5.2 Comparison of Models

A comparative analysis was generated in a Pandas DataFrame at the end of the notebook. The results on the testing data are as follows:

Model	RMSE	MAE	R^2 Score
Linear Regression	~72,688	~51,000	~0.612
Random Forest	~49,455	~32,900	~0.817
XGBoost	48,496.32	32,452.53	0.821



5.3 Best-Performing Model and Interpretation

Best Model: The XGBoost Regressor emerged as the definitive winner.

Interpretation:

- The Linear Regression model underfitted the data, yielding an R^2 of only ~0.61. This proves that house prices do not scale in a strictly straight line with features like income or location.
- Random Forest made a massive leap in accuracy (R^2 ~0.817), successfully capturing non-linear relationships.
- XGBoost slightly edged out Random Forest, achieving the lowest MAE ($32,452.53) and the highest R^2 (0.821). This means that on average, the XGBoost model's predictions are off by about $32,452, which is highly acceptable given that house prices span up to $500,001. The gradient boosting mechanism effectively minimized the residual errors left by initial trees.

Chapter 6: Conclusion and Future Scope

6.1 Summary of Work

This project successfully implemented an end-to-end Machine Learning pipeline to predict house prices in California. The raw data was visualized, revealing anomalies like price caps. A robust preprocessing pipeline was constructed using Scikit-Learn to impute missing bedrooms and scale numerical figures, preventing data leakage. Three models were trained and rigorously evaluated.

6.2 Achievements

- Successfully prevented data leakage through proper Pipeline architecture.
- Replaced missing values and encoded categorical geography seamlessly.
- Achieved a high predictive accuracy with an R^2 Score of 0.821 using the XGBoost Regressor, demonstrating the model's strong capability to understand real estate data.

6.3 Limitations

- Data Capping: The target variable (median_house_value) and housing_median_age were capped at $500,001 and 52 years, respectively. The model learned this artificial ceiling, which may cause it to severely under-predict the value of ultra-luxury homes.
- Static Dataset: The dataset represents a specific historical snapshot and does not account for modern inflation, changing interest rates, or current housing market crashes.

6.4 Future Improvements

- Outlier Removal: Removing the capped data points ($500,001) during preprocessing could help the model learn the true upper-bound trajectory of house prices.
- Hyperparameter Tuning: Implementing GridSearchCV or RandomizedSearchCV on the XGBoost model to find the mathematically optimal max_depth and learning_rate.
- Geospatial Feature Engineering: Using the latitude and longitude to calculate the distance to major economic hubs (like Silicon Valley or Los Angeles) rather than relying solely on ocean_proximity.
- Deployment: Wrapping the model in a FastAPI or Streamlit backend to serve predictions via a user-friendly web interface.

References

20. A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
21. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
22. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785-794.
23. L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.
24. W. McKinney, "Data Structures for Statistical Computing in Python," in Proceedings of the 9th Python in Science Conference, Austin, TX, 2010, pp. 51-56.
25. J. D. Hunter, "Matplotlib: A 2D Graphics Environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90-95, 2007.
26. M. Waskom, Seaborn: Statistical Data Visualization, [Online]. Available: https://seaborn.pydata.org/.
27. R. Pace pace and R. Barry, "Sparse Spatial Autoregressions," Statistics & Probability Letters, vol. 33, no. 3, pp. 291-297, 1997.

Appendix

1. Project Summary

This project focuses on predicting California housing prices using demographics and location data. By implementing an automated Scikit-Learn preprocessing pipeline to impute missing values and encode data, the workflow ensures mathematical stability and prevents data leakage. Linear Regression, Random Forest, and XGBoost were compared. XGBoost emerged as the best model, achieving an R^2 score of 0.821 and an MAE of ~32,452, proving gradient boosting's superiority on tabular real-estate data.

2. Technologies Used

- Programming Language: Python 3.x
- Development Environment: Jupyter Notebook
- Data Manipulation: Pandas, NumPy
- Data Visualization: Matplotlib, Seaborn
- Machine Learning Frameworks: Scikit-Learn, XGBoost

3. Flowchart of Project Workflow

